

Appendix: Minimising Edge Additions for Joint k -Core Activation of Query Nodes

Juhyeok Seo^{1,*}, Taejoon Han^{1,*}, Seungjoo Jung², Junghoon Kim^{1,†}
¹Ulsan National Institute of Science and Technology (UNIST), ²Ajou University

1. 정리1의 증명

주어진 무방향 그래프 $G = (V, E)$, 쿼리 노드 집합 $Q \subseteq V$, 정수 $k \geq 4$, 그리고 예산 b 에 대하여, $|A| \leq b$ 인 간선 집합

$$A \subseteq \binom{V}{2} \setminus E$$

를 추가하여 모든 쿼리 노드가 k -core에 포함되도록 할 수 있는지를 결정하는 문제를 고려한다.

정리 1. 임의의 정수 $k \geq 4$ 에 대하여 위 결정 문제는 NP-complete이며, 따라서 최소 비용 쿼리 집합 k -core 공동 활성화 문제는 NP-hard이다.

증명 잘 알려진 NP-complete 문제인 Set Cover [1]로부터 다항시간 환원을 보인다.

Set Cover의 인스턴스 $I_{SC} = (I, S, b')$ 에서

$$I = \{i_1, \dots, i_m\}, \quad S = \{S_1, \dots, S_n\},$$

그리고 양의 정수 b' 라 하자. 여기서 각 $S_j \subseteq I$ 이다. Set Cover의 결정 문제는 다음 조건을 만족하는 $S' \subseteq S$ 가 존재하는지를 묻는다.

$$|S'| \leq b', \quad \bigcup_{S_j \in S'} S_j = I.$$

일반성을 잃지 않고 $\bigcup_{S_j \in S} S_j = I$ 라고 가정한다.

이제 주어진 I_{SC} 로부터 최소 비용 쿼리 집합 k -core 공동 활성화 문제의 인스턴스 $I_{MQ} = (G, Q, k, b)$ 를 다음과 같이 구성한다.

기본 구조. 최소 비용 쿼리 집합 k -core 공동 활성화 문제의 임계값과 예산을 각각

$$k = |I| + 4, \quad b = b'$$

로 설정한다.

다음으로 $k + 1$ 개의 노드로 이루어진 clique B 를 만든다. B 의 모든 노드는 clique 내부에서 차수 k 를 가지므로 항상 k -core에 남는다.

쿼리 구조. 각 원소 $i_r \in I$ 에 대해 쿼리 노드 q_r 와 두 보조 노드 x_r, y_r 를 만든다. q_r 를 x_r 와 y_r 에 각각 연결하고, q_r, x_r, y_r 를 각각 B 의 임의의 $k - 2$ 개 노드와 연결한다.

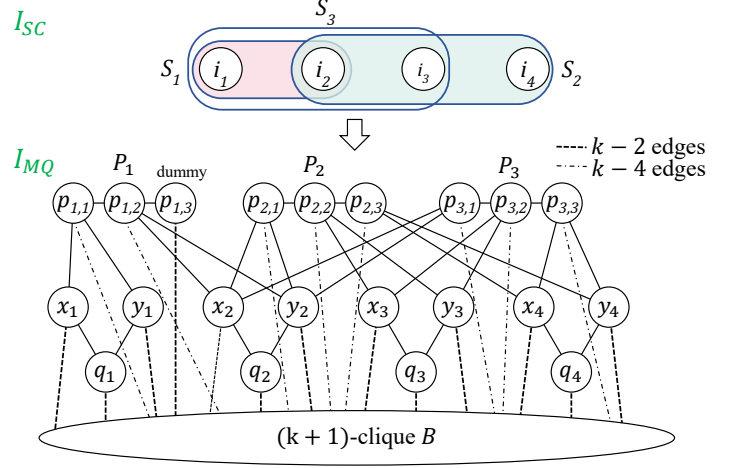


그림 1: Gadget 예시

쿼리 집합은

$$Q = \{q_1, \dots, q_m\}$$

이다.

집합 구조. 각 집합 $S_j \in S$ 에 대해

$$\ell_j = \max\{|S_j|, 3\}$$

개의 노드로 이루어진 경로

$$P_j : p_{j,1} - p_{j,2} - \dots - p_{j,\ell_j}$$

를 만든다.

각 $i_r \in S_j$ 에 서로 다른 하나의 경로 노드 $p_{j,t}$ 를 배정하고,

$$(p_{j,t}, x_r), \quad (p_{j,t}, y_r)$$

를 추가한다.

원소가 배정된 경로 노드 $p_{j,t}$ 는 B 의 임의의 $k - 4$ 개 노드와 연결한다. $|S_j| < 3$ 인 경우에는 원소가 대응되지 않은 경로 노드가 존재하며, 이를 dummy 노드라 하고 B 의 임의의 $k - 2$ 개 노드와 연결한다.

집합 S_j 를 선택하는 것은

$$e_j = (p_{j,1}, p_{j,\ell_j})$$

를 추가하여 P_j 를 cycle로 만드는 것에 대응한다. 이에 대한 성질은 이후 증명에서 보인다. 그림 1은 제안한 환원 과정의 gadget

[†]Corresponding Author.

^{*}Equal contributions.

예시를 보여준다.

기본 그래프의 k -core 분해. 어떠한 간선 e_j 도 추가되지 않은 경우를 고려하자. 각 경로 P_j 의 양 끝 노드는 원소가 대응된 경우

$$(k-4) + 1 + 2 = k-1$$

개의 이웃을 가지며, dummy 노드인 경우에도

$$(k-2) + 1 = k-1$$

개의 이웃만을 가진다. 따라서 k -core 분해 과정에서 경로의 양 끝 노드가 먼저 제거된다. 이후 새롭게 양 끝이 된 노드들도 동일한 이유로 제거되며, 이 과정이 반복되어 모든 P_j 의 노드가 제거된다.

모든 경로 노드가 제거되면 x_r 와 y_r 는 각각

$$(k-2) + 1 = k-1$$

개의 이웃만을 가지므로 제거되고, 이어서 q_r 역시 제거된다. 따라서 추가 간선이 없는 기본 그래프의 k -core에는 clique B 의 노드만 남는다.

반면 $e_j = (p_{j,1}, p_{j,\ell_j})$ 를 추가하면 P_j 가 cycle을 형성하여 위의 연쇄적인 제거가 발생하지 않는다. 따라서 e_j 의 추가는 집합 S_j 를 활성화하는 것에 대응한다.

이제 두 문제의 해가 서로 대응함을 정방향과 역방향으로 각각 보인다.

정방향. 크기 b' 이하의 Set Cover $S' \subseteq S$ 가 존재한다고 하자. 각 $S_j \in S'$ 에 대해 e_j 를 추가한다. 그러면 선택된 집합의 경로 P_j 는 cycle이 된다.

각 노드의 차수를 확인하면 다음과 같다.

- 원소가 대응된 경로 노드 $p_{j,t}$ 는

$$(k-4) + 2 + 2 = k$$

개의 이웃을 가진다. 여기서 두 개는 cycle 상의 이웃이고, 나머지 두 개는 x_r 와 y_r 이다.

- dummy 경로 노드는

$$(k-2) + 2 = k$$

개의 이웃을 가진다.

- $i_r \in S_j$ 인 경우, x_r 와 y_r 는 각각

$$(k-2) + 1 + 1 = k$$

개의 이웃을 가진다. 여기서 $k-2$ 개는 B 의 노드이고, 나머지는 q_r 와 대응되는 $p_{j,t}$ 이다.

- 따라서 x_r 와 y_r 가 모두 k -core에 남으며, q_r 역시

$$(k-2) + 1 + 1 = k$$

개의 이웃을 가지므로 k -core에 포함된다.

따라서 S' 가 모든 원소를 cover하면 모든 $q_r \in Q$ 가 k -core에 포함된다. 또한 추가한 간선의 수는

$$|S'| \leq b' = b$$

이므로 유효한 해가 존재한다.

최적 간선 선택의 성질. 간선 수를 최소화하는 최적해를 A^* 라 하자. 우리는 A^* 와 동일한 수의 간선을 사용하면서, 집합 구조를 활성화하는 간선 e_j 들로만 구성된 최적해가 항상 존재함을 보인다.

먼저 집합 경로 P_j 의 일부 노드가 k -core에 남기 위해서는 해당 경로의 양 끝에서 부족한 차수를 보완해야 하므로 적어도 두 개의 추가 간선 끝점이 필요하다. 반면

$$e_j = (p_{j,1}, p_{j,\ell_j})$$

하나를 추가하면 동일한 하나의 간선 비용으로 P_j 전체를 cycle로 만들 수 있으며, 이에 따라 S_j 에 속한 모든 원소의 쿼리 구조를 동시에 활성화할 수 있다.

또한 어떠한 집합 경로의 도움 없이 쿼리 q_r 를 직접 활성화하는 경우에도 q_r , x_r , y_r 의 부족한 차수를 보완하기 위해 적어도 하나의 간선에 해당하는 추가 지지가 필요하다. 이때 $i_r \in S_j$ 인 임의의 집합 S_j 에 대해 e_j 를 선택하면 동일한 하나의 간선으로 q_r 를 활성화할 수 있으며, 동시에 S_j 에 포함된 다른 쿼리들도 활성화할 수 있다.

따라서 집합 구조 외부의 간선을 사용하는 최적해가 존재하더라도, 해의 크기를 증가시키지 않고 해당 간선들을 적절한 e_j 들로 치환할 수 있다. 그러므로 e_j 들로만 구성된 최적해 A_E^* 가 존재하며,

$$|A_E^*| = |A^*|.$$

역방향. $|A| \leq b$ 인 유효한 해가 존재한다고 하자. 그러면 최적해 A^* 에 대해

$$|A^*| \leq b$$

이다.

앞서 보인 성질에 의해 e_j 들로만 구성된 최적해 A_E^* 가 존재한다. 이에 대응하여

$$S' = \{S_j \mid e_j \in A_E^*\}$$

라 하자.

모든 쿼리 q_r 가 k -core에 포함되어 있으므로, 각 원소 $i_r \in I$ 에 대해 $i_r \in S_j$ 이고 $e_j \in A_E^*$ 인 집합 S_j 가 적어도 하나 존재한다.

따라서

$$\bigcup_{S_j \in S'} S_j = I.$$

또한

$$|S'| = |A_E^*| = |A^*| \leq b.$$

따라서 S' 는 크기 b 이하의 Set Cover이다.

구성되는 그래프의 크기는 Set Cover 인스턴스 크기에 대해 다항식이다. 또한 주어진 A 에 대해 k -core는 표준 peeling 알고리즘으로 다항시간에 계산할 수 있으므로 결정 문제는 NP에 속한다.

따라서 위 결정 문제는 NP-complete이다. 또한 최소 비용 쿼리 집합 k -core 공동 활성화 최적화 문제를 다항시간에 해결할 수 있다면, 그 최적값이 b 이하인지 확인함으로써 위 결정 문제 역시 다항시간에 해결할 수 있다. 따라서 이에 대응하는 최적화 문제는 NP-hard이다. $\square$

2. Pseudo Code: RandomSupportEdges

Algorithm 1: RANDOMSUPPORTEDGES

Input: S^* : selected candidate, G_A : current graph, $C_k(G_A)$: current k -core, Q : query set, k : integer

Output: ΔA : added edge set

```

1  $\Delta A \leftarrow \emptyset$ ;
2  $U \leftarrow S^* \cup Q \cup C_k(G_A)$ ;
3 foreach  $v \in S^*$  do
4    $d(v) \leftarrow \max\{0, k - \deg(v, U)\}$ ;
5 while  $\exists u, v \in S^*$  such that  $u \neq v, d(u) > 0, d(v) > 0$ , and
    $(u, v) \notin E(G_A) \cup \Delta A$  do
6    $P \leftarrow \{(u, v) \mid u, v \in S^*, u \neq v, d(u) > 0, d(v) > 0, (u, v) \notin E(G_A) \cup \Delta A\}$ ;
7   Select  $(u, v)$  uniformly at random from  $P$ ;
8    $\Delta A \leftarrow \Delta A \cup \{(u, v)\}$ ;
9    $d(u) \leftarrow d(u) - 1$ ;
10   $d(v) \leftarrow d(v) - 1$ ;
11  $Q' \leftarrow Q \setminus C_k(G_A)$ ;
12 foreach  $v \in S^*$  such that  $d(v) > 0$  do
13   while  $d(v) > 0$  do
14     Select  $q \in Q'$  uniformly at random such that
        $(q, v) \notin E(G_A) \cup \Delta A$ ;
15      $\Delta A \leftarrow \Delta A \cup \{(q, v)\}$ ;
16      $d(v) \leftarrow d(v) - 1$ ;
17 return  $\Delta A$ ;
```

Algorithm 1은 선택된 후보 S^* 의 노드들이 k -core에 포함되기 위해 필요한 구조적 지지를 만족하도록 추가 간선 집합 ΔA 를 생성한다. 먼저 S^* , 쿼리 집합 Q , 현재 k -core $C_k(G_A)$ 로 구성된 집합 U 를 정의하고, 각 $v \in S^*$ 에 대하여 부족한 지지량 $d(v) = \max\{0, k - \deg(v, U)\}$ 를 계산한다 (lines 1-4). 이후 지지량이 부족한 무작위 두 후보 노드가 서로 연결 가능하면 이들 사이에 간선을 추가한다. 이때 P 는 연결 가능한 모든 간선 집합이다 (lines 5-10). 더 이상 후보 내부에서 연결할 수 없는 경우에는 아직 활

성화되지 않은 쿼리 집합 $Q' = Q \setminus C_k(G_A)$ 을 구성하고, 남아 있는 부족량을 갖는 후보 노드와 쿼리 노드를 연결하여 필요한 지지를 보완한다 (lines 11-16). 최종적으로 생성된 간선 집합 ΔA 를 반환한다.

3. Pseudo Code: GAIN

Algorithm 2: GAIN

Input: G : graph, Q : query set, k : integer

Output: A : added edge set

```

1  $A \leftarrow \emptyset$ ;
2  $C_k(G_A) \leftarrow \text{FIND}k\text{-CORE}(G_A, k)$ ;
3 while  $Q \not\subseteq C_k(G_A)$  do
4    $G_A \leftarrow (V, E \cup A)$ ;
5    $C_k(G_A) \leftarrow \text{FIND}k\text{-CORE}(G_A, k)$ ;
6    $Q' \leftarrow Q \setminus C_k(G_A)$ ;
7   foreach  $q \in Q'$  do
8      $R_A(q) \leftarrow \max\{0, k - |N(q, G_A) \cap C_k(G_A)|\}$ ;
9    $S^* \leftarrow \emptyset$ ;
10   $\text{score}_Q^* \leftarrow 1$ ;
11   $S \leftarrow \text{CONNECTEDCOMP}(G_A, (C_{k-1}(G_A) \setminus C_k(G_A)) \setminus Q)$ ;
12  foreach  $S_i \in S$  do
13     $(L_1, \dots, L_{\ell_i}) \leftarrow \text{ONIONLAYERS}(S_i, k)$ ;
14     $\text{prevCost} \leftarrow +\infty$ ;
15    for  $j \leftarrow 1$  to  $\ell_i$  do
16       $S_i^j \leftarrow \bigcup_{r=j}^{\ell_i} L_r$ ;
17       $U \leftarrow S_i^j \cup Q \cup C_k(G_A)$ ;
18       $NS(S_i^j) \leftarrow \sum_{v \in S_i^j} \max\{0, k - \deg(v, U)\}$ ;
19      if  $NS(S_i^j) \geq \text{prevCost}$  then
20        break;
21       $G_Q(S_i^j) \leftarrow \sum_{q \in Q'} \min\{R_A(q), \sum_{v \in S_i^j} c_A(v, q)\}$ ;
22       $\text{score}_Q \leftarrow G_Q(S_i^j) / NS(S_i^j)$ ;
23       $\text{prevCost} \leftarrow NS(S_i^j)$ ;
24      if  $\text{score}_Q > \text{score}_Q^*$  then
25         $\text{score}_Q^* \leftarrow \text{score}_Q$ ;
26         $S^* \leftarrow S_i^j$ ;
27  if  $S^* \neq \emptyset$  then
28     $\Delta A \leftarrow \text{RANDOMSUPPORTEDGES}(S^*, G_A, C_k(G_A), Q, k)$ ;
29  else
30     $\Delta A \leftarrow \text{QUERY}(C_k(G_A), Q, k)$ ;
31     $A \leftarrow A \cup \Delta A$ ;
32     $\Delta A \leftarrow k\text{-CORE}(C_k(G_A), Q, k)$ ;
33   $A \leftarrow A \cup \Delta A$ ;
34 return  $A$ ;
```

Algorithm 2는 GAIN의 전체 과정을 나타낸다. 이 과정에서 사용되는 함수 FIND k -CORE와 ONIONLAYERS는 그래프와 k 를 받아 각각 k -core 그래프를 찾고 onionlayer를 구분해서 반환하며, CONNECTEDCOMP는 그래프와 노드 집합을 받아 해당 노드 집합으로 유도하는 연결 성분들을 반환한다. QUERY와 k -CORE는 그래프, 쿼리 집합, k 를 받아서 모든 쿼리가 k -core에 포함되거나

더 이상 연결이 불가능할 때까지 각각 쿼리끼리, 쿼리와 k -core 사이에 간선을 추가하고 추가한 간선집합을 반환한다. 모든 쿼리가 k -core에 포함될 때까지 다음 과정을 반복한다 (line 3). 현재 그래프 G_A 와 $C_k(G_A)$ 를 갱신하고, 아직 활성화되지 않은 쿼리 Q' 에 대해 부족량 $R_A(q)$ 를 계산한다 (line 4-8). 또한 매 반복마다 $S^* \leftarrow \emptyset$, $score_Q^* \leftarrow 1$ 로 초기화한다. 쿼리끼리 연결할 경우 $score_Q^*$ 가 1과 같으므로 이 값을 1로 초기화한다 (line 9-10). 이후 $(C_{k-1}(G_A) \setminus C_k(G_A)) \setminus Q$ 가 유도하는 연결 성분을 후보 서브그래프 집합 S 로 구성한다 (line 11). 각 후보 S_i 에 대해서는 onion layer $L_1, \dots, L_{\ell_i}$ 를 구하고, 바깥쪽 layer를 순차적으로 제외하여 $S_i^j = \bigcup_{r=j}^{\ell_i} L_r$ 를 구성한다 (line 12-16). 각 S_i^j 에 대해 필요한 구조적 지지 $NS(S_i^j)$ 를 계산하며, 그 값이 직전에 계산한 후보의 지지량보다 감소하지 않는 경우 더 효율적일 수는 없으므로 이후 탐색을 조기에 종료한다 (line 17-20). 그렇지 않은 경우 쿼리 부족량의 감소를 나타내는 $G_Q(S_i^j)$ 를 계산하고, 이를 필요한 지지량으로 나눈 $score_Q$ 를 이용하여 후보의 효율을 평가한다 (line 21-23). 현재까지의 최고 점수보다 높은 후보가 발견되면 S^* 와 $score_Q^*$ 를 갱신한다 (line 24-26). 유효한 후보 S^* 가 선택되면 Algorithm 1의 RANDOMSUPPORTEDEDGES를 이용하여 후보 활성화에 필요한 간선을 생성한다 (line 27-28). 반면 조건을 만족하는 후보가 존재하지 않으면 주변 후보를 이용한 활성화 대신 QUERY를 통해 남은 쿼리 사이의 연결을 우선 추가하고, 그래도 쿼리가 k -core에 포함되지 못하는 경우를 방지하기 위해 k -CORE를 통해 지지가 필요한 쿼리를 현재 k -core와 직접 연결한다 (line 29-32). 생성된 간선을 A 에 추가한 뒤 변경된 그래프를 기준으로 다음 반복을 수행하며, 최종적으로 $Q \subseteq C_k(G_A)$ 가 만족되면 추가 간선 집합 A 를 반환한다 (line 33-34).

참고 문헌

- [1] R. M. Karp, "Reducibility among combinatorial problems," in *Complexity of Computer Computations*, pp. 85–103, Springer, 1972.